Applied Human Computer Interaction (AHCI)

Asif Fazwani

Chapter 3 – The Interaction

User–Computer Communication

Why Interaction Matters

- Users must **communicate their requirements** to the system
- The computer must **respond appropriately**

This communication happens through:

- Input (clicks, taps, typing, voice)
- Output (screens, sound, vibration, animations)

Example:

- Sending a voice note on WhatsApp
- Tapping reactions on Instagram stories

Low Interaction – Batch Input

Batch Input

- User provides **all input at once**
- Computer performs task **without further user involvement**
- Very limited interaction

Problems

- No immediate feedback
- Poor support for many modern tasks

Example:

- Uploading a form and waiting hours/days for a response
- Submitting an assignment portal that shows no status update

High Interaction – Interactive Systems

Highly Interactive Systems

- User gives instructions **continuously**
- System provides **instant feedback**
- Interaction is dynamic and responsive

Example:

- Dragging files on a laptop
- Zooming photos with pinch gesture
- Playing VR games or using Meta Quest

The Interaction

- **Interaction Models:** Conceptual frameworks to explain how users and systems interact.
  Understanding how a user: Thinks → taps → sees feedback on Instagram
- **Ergonomics:** Designing systems that fit human physical abilities
  One-handed phone usage, Dark mode to reduce eye strain
- **Interaction styles:** The nature of user/system dialog It can be Simple (tap → response) or Complex (multi-step workflows)
  Voice commands to Siri / Google Assistant, Drag-and-drop in Canva
- **Context:** social, organizational (Social: Group chats, shared Google Docs) (Organizational: University LMS, internship portals)

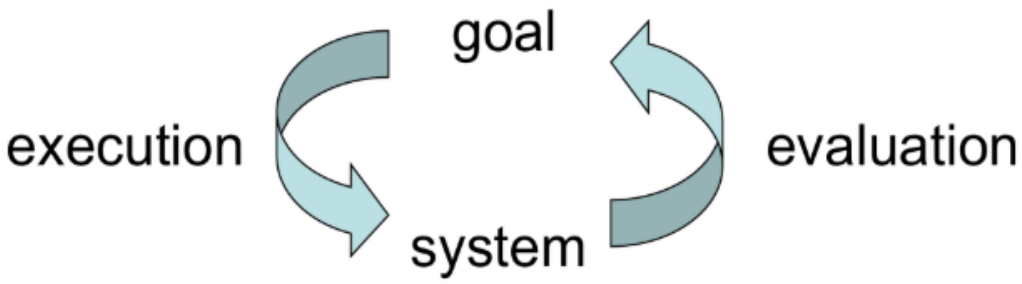
Some Terms of Interaction

- **Domain**
  the area of work under study (*graphic design)*
- **Goal**
  – what you want to achieve (*create a solid red triangle)*
- **Task**
  – how you go about doing it
  – ultimately in terms of operations or actions (*select fill tool, click over triangle)*
- **Note**
  – traditional interaction
  – use of terms differs a lot
  – especially **task vs goal** !!

Norman's Model of Interaction

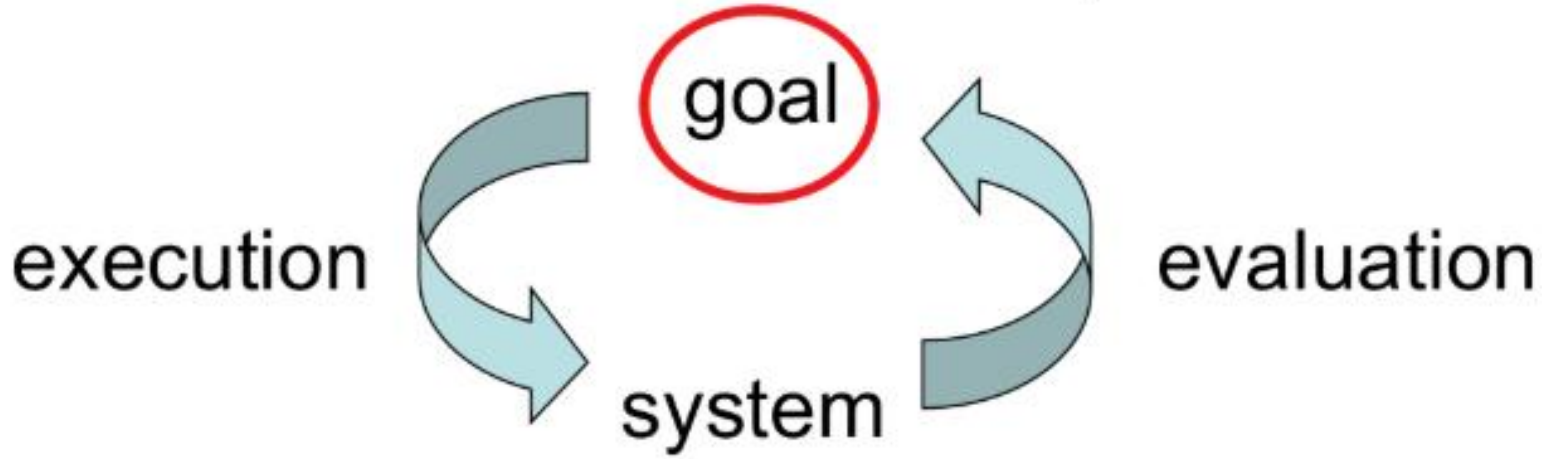
- One of the most influential models in HCI
- Closely matches our intuitive understanding of interaction
- Describes interaction as a cycle between user and computer

execution/evaluation loop



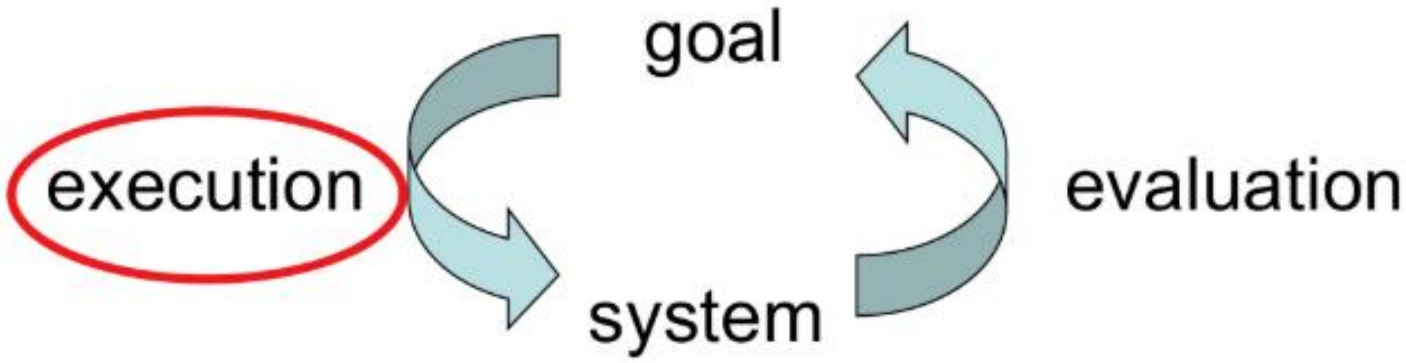
- user establishes the goal
- formulates intention
- specifies actions at interface
- executes action
- perceives system state
- interprets system state
- evaluates system state with respect to goal

execution/evaluation loop



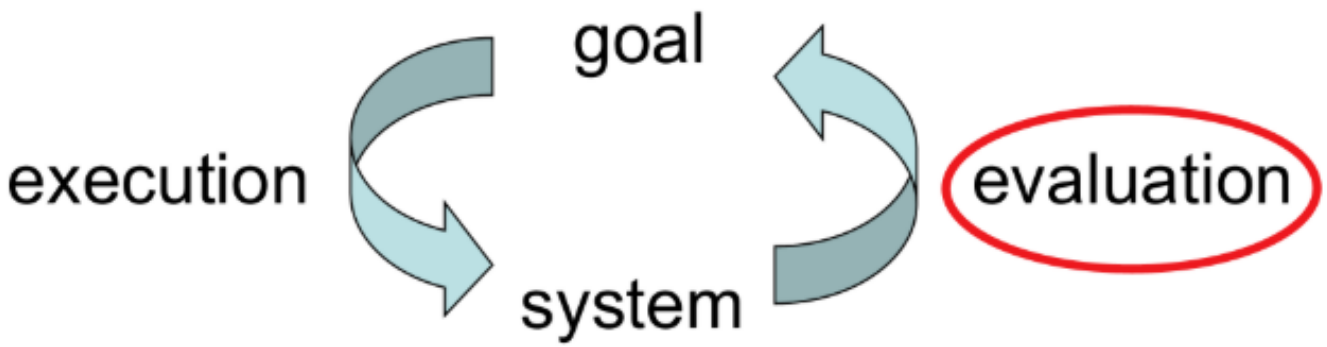
- user establishes the goal
- formulates intention
- specifies actions at interface
- executes action
- perceives system state
- interprets system state
- evaluates system state with respect to goal

execution/evaluation loop



- user establishes the goal
- formulates intention
- specifies actions at interface
- executes action
- perceives system state
- interprets system state
- evaluates system state with respect to goal

execution/evaluation loop



- user establishes the goal
- formulates intention
- specifies actions at interface
- executes action
- perceives system state
- interprets system state
- evaluates system state with respect to goal

Example: Sending an Email

1. **Establishing the goal**
   – User decides what needs to be achieved
   → *Send an email to the teacher*
2. **Forming the intention**
   – Decide how to achieve the goal
   → *Use the email application to send the message*
3. **Specifying the action sequence**
   – Plan the exact actions
   → *Open email app → click "Compose" → enter address → type message → click "Send"*

Example: Sending an Email

4. Executing the action
– Carry out the planned actions
→ *User performs the steps at the interface*

5. Perceiving the system state
– Observe what the system shows
→ *Message like “Email sent” appears*

6. Interpreting the system state
– Understand what the system state means
→ *The email has been successfully delivered*

7. Evaluating the system state
– Compare outcome with original goal
→ *Goal achieved: email has been sent*

Example: Sending an Email

If the email is not sent

- User interprets the state (e.g. error message)
- Forms a **new goal or intention**
- Repeats the interaction cycle

Example: Switching on a Light

Imagine you are sitting reading as evening falls. You decide you need more light; that is you establish the goal to get more light.

Different Intentions (same goal)

- Intend to **switch on the desk lamp**
- Intend to **switch on the ceiling light**
- Intend to **ask someone else to turn on the light**
- Intend to **open the curtains** (if daylight is available)

Goal is the same, but **intentions differ**

Different Actions (based on intention)

- **Intention: switch on desk lamp**
- Reach for lamp, Press lamp switch
- **Intention: switch on ceiling light**
- Walk to wall switch, Flip switch
- **Intention: ask someone else**
- Call out to another person, Wait for them to act
- **Intention: open curtains**
- Stand up, Pull curtains open

After Action

- User perceives the result (light on or not)
- Interprets the result (enough light / not enough)
- Evaluates whether the goal is achieved
- If not → new intention and actions are formed

This shows:

- **Goal ≠ intention ≠ action**
- Multiple interaction paths can exist for the same goal

Gulf of Execution

What it is

- The gap between **what the user wants to do** and **what actions the system allows**
- In simple words: *“How do I do what I want to do?”*

Example: Switching on a Light

- **User's goal:** Get more light
- **User's intention:** Switch on the lamp
- **Problem (Gulf of Execution):**
- Switch is hidden, oddly shaped, or unclear
- User does not know **which switch** controls the lamp
- The user's intended action does **not match** the actions clearly supported by the system
  Reduce gulf of execution:
- Switch is visible, labeled, easy to press
- User immediately knows what to do

Gulf of Evaluation

What it is

- The gap between **what the system shows** and **what the user expects or understands**
- In simple words: *“Did it work? What happened?”*

Example: Switching on a Light

- **After action:** User presses the switch
- **Problem (Gulf of Evaluation):**
- Light does not turn on
- No clear indication why
- User wonders:
- Is the bulb blown?
- Is there no power?
- Did I press the right switch?
- User finds it hard to **interpret the system state**
  Reduce gulf of evaluation:
- Light turns on immediately
- Or a clear indicator shows what went wrong

Conclusion

Good interface design aims to **reduce both gulfs** so that users can easily act and easily understand the results

Abowd and Beale Framework

Abowd and Beale **extend Norman's model** by explicitly showing **where translation happens** in interaction.

They divide interaction into four components:

- **User**
- **Input**
- **System**
- **Output**

Each component uses a **different language**.

The Four Parts with Languages

1. User (Task Language)

- Describes goals and intentions
- Based on human thinking and psychology

Example:
“I want to send a message”
“I want more light”

The Four Parts with Languages

2. Input (Input Language)

- How the user expresses intentions to the system
- Physical actions at the interface

Example:

- Typing on keyboard
- Clicking a button
- Pressing a switch

The Four Parts with Languages

3. System (Core Language)

- Internal system representation
- Data structures, variables, states

Example:

- message_status = sent
- light_state = ON

The Four Parts with Languages

4. Output (Output Language)

- How the system presents its state
- What the user sees/hears/feels

Example:

- “Message sent” notification
- Light turning on
- Status icon changing

Using Abowd & Beale’s model

user intentions

→ translated into actions at the interface

→ translated into alterations of system state

→ reflected in the output display

→ interpreted by the user

Example : Switching on a Light

User

Intention: *Get more light*

Input

Action: *Press the wall switch*

System

State change: *Electric circuit closes*

Output

Light turns ON

User Interpretation

"The room is bright enough"

Goal achieved → interaction complete

Ergonomics

Ergonomics is the study of the **physical characteristics of interaction** between users and systems.

It focuses on:

- How **controls are designed**
- The **physical environment** of interaction
- The **layout and physical qualities of the screen**

Main Focus of Ergonomics

- **User performance**
- How the interface:
  - Enhances performance, or
  - Detracts from performance

Good ergonomics → faster, safer, more comfortable interaction
Poor ergonomics → errors, fatigue, discomfort

Issues Addressed by Ergonomics

1. Arrangement of Controls and Displays

- Placement of buttons, switches, menus, icons
- Grouping and spacing for easy access

Example: Frequently used buttons placed within easy reach

2. Physical Environment

- Lighting, Noise, Temperature, Workspace setup

Example: Poor lighting causing eye strain while using a screen

Issues Addressed by Ergonomics

3. Health Issues

- Repetitive strain injuries
- Eye strain
- Posture-related problems

Example: Long hours of keyboard use causing wrist pain

4. Use of Color

- Visibility
- Readability
- Avoiding confusion or strain

Example: Low-contrast text making information hard to read

Glass Interfaces vs Dials and Knobs

Traditional Interfaces: Dials and Knobs

- Physical controls
- Directly **wired or piped** to equipment
- Information is:
  - Physically located
  - Continuously visible
- Easy to read at a glance
- Provide **strong context cues** (position, grouping, location)

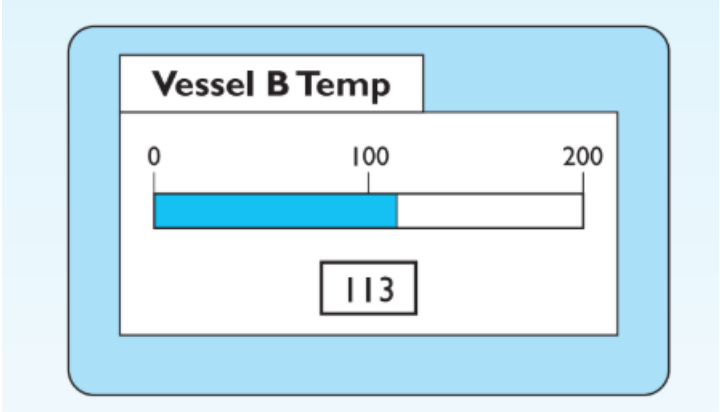
Example:

- Pressure gauge
- Temperature dial
- Mechanical knobs on machinery

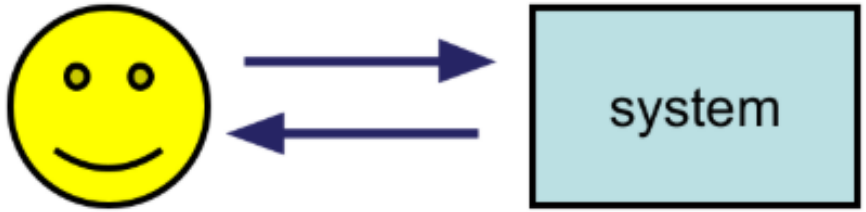
Glass Interfaces vs Dials and Knobs

Glass Interfaces

- Computer-based displays (screens)
- Used to **monitor and control equipment**
- Replace some or all physical controls
- Common in:
  - Control rooms
  - Aircraft cockpits
  - Power plants



Direct manipulation



What's happening here:

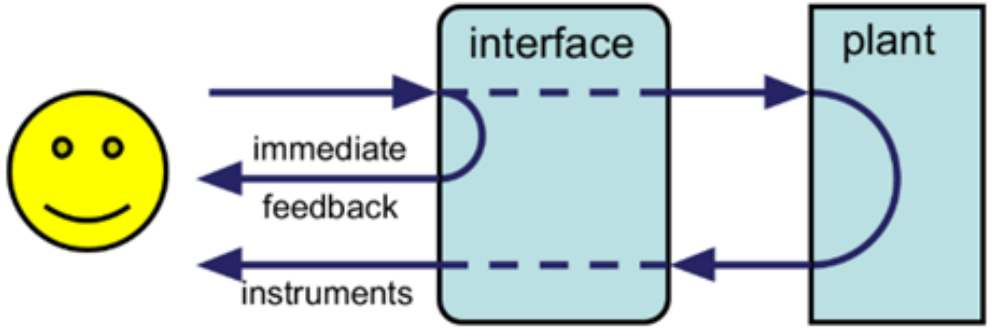
- The **user directly manipulates objects inside the computer**.
- Example: dragging a file, resizing a window, editing text in Word.

Characteristics:

- The computer's world *is* the world the user is acting on.
- Actions have **immediate and visible effects**.
- The user is in **primary control** at all times.
- Feedback is instant and unambiguous (you drag → the icon moves).

Mental model: I act on the screen, and the screen changes right away.

Indirect manipulation



What's happening here:

- The user is **not interacting with the real system directly**.
- The interface sits in between the **operator** and the **plant** (the real-world machinery or process).
- The interface is just a **mediator**, not the thing being controlled.

Mental model:

I tell the system what to do, and later I see what really happened.

Indirect manipulation Example

Power grid control (SCADA systems):

What the operator does

- Opens/closes breakers
- Adjusts voltage or load levels via a control panel

Why it's indirect

- The operator isn't touching the grid itself
- Commands travel through networks to substations
- Feedback comes later from sensors

Typical issue

- A breaker may show "open" on the screen before the physical mechanism fully moves
- Sensor data may be delayed or sampled

Common interaction styles

- command line interface
- menus
- natural language
- question/answer and query dialogue
- form-fills and spreadsheets
- WIMP
- point and click
- three–dimensional interfaces

Command Line Interface (CLI)

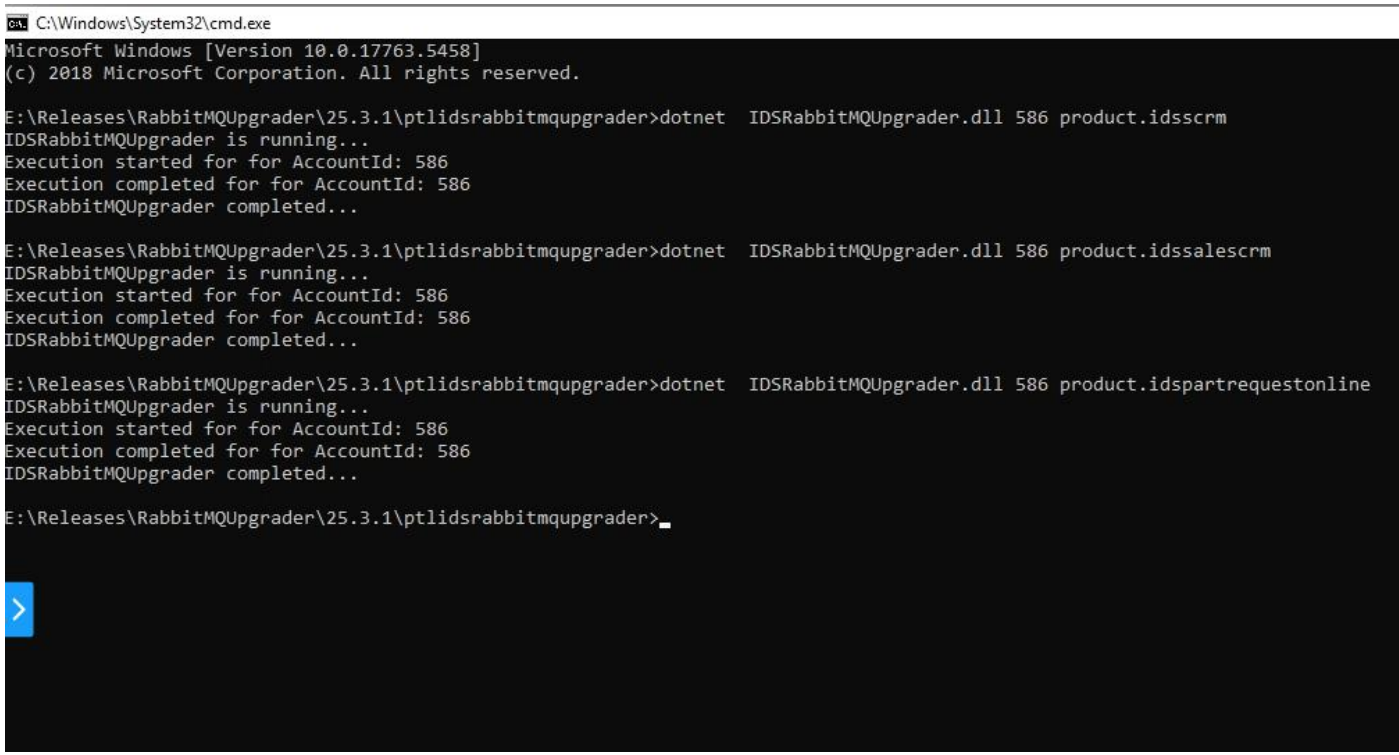
- **Interaction Style:** Users interact by **typing commands** (words, abbreviations, function keys)
- Gives **direct & fast access** to system functions
- Very **powerful and flexible** (combine commands, repeat tasks quickly)
- **No visual cues** → commands must be **memorized**
- Best suited for **expert users**, difficult for beginners

Examples:

- Terminal on **Mac / Linux**
- **Git commands** (git clone, git push)
- **Windows PowerShell**
- Using **npm / Python commands** for coding & AI projects

Limitation:

- Commands differ across systems → **high learning effort**



Menu-Driven Interface

- User selects from **visible options** on the screen
- Uses **recognition over recall** (easier than remembering commands)
- Options must be **clearly named & logically grouped**
- Often **hierarchical** (menus → sub-menus)
- Can be **text-based** or **graphical**
- Less powerful than CLI but **easy for beginners**

Examples:

- **Instagram settings menus**
- **ATM / UPI app options**
- **Game menus** (Start, Settings, Load)
- **Food delivery apps** (categories → items)

Natural Language Interface

- Users interact using everyday language (text or speech)
- Feels easy and natural for users
- Major challenge: ambiguity Same sentence or word can have multiple meanings
- Computers struggle with context, pronouns & vague terms.

Examples:

- ChatGPT prompts
- Siri / Alexa / Google Assistant
- Voice search: *“Play chill music”*
- Customer support chatbots

Query interfaces

- Used to **search databases**
- Looks like natural language but needs **specific syntax**
- Becomes complex with **multiple conditions**
- Requires **experience**

Examples

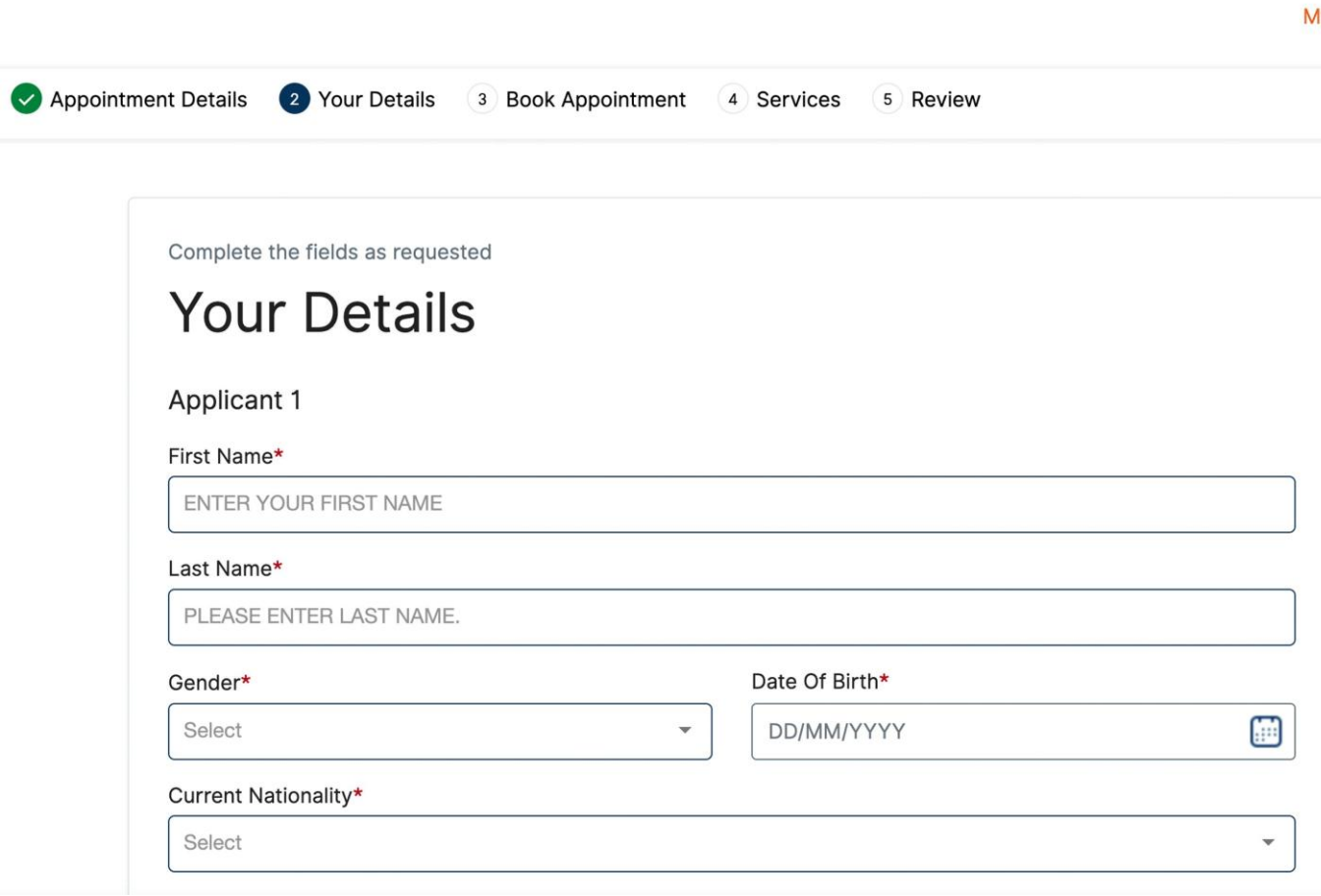
- **Google search with filters**
- **E-commerce filters** (price, brand, rating)
- **SQL queries** in projects

Form-Fills

- Screen looks like a **paper form**
- User fills in **fields / slots**
- Easy to **learn and use**
- Supports **editing, skipping fields, corrections**
- Best for **data entry** and **novice users**

Examples:

- Online registration forms
- Google Forms
- Job / internship applications
- UPI or banking forms



Spreadsheets

- Grid of **cells** with values or **formulae**
- Automatic **updates & consistency**
- User can **experiment with values**
- **Input and output are mixed** (very flexible)

Examples

- Google Sheets / Excel
- Budget tracking
- Marks / CGPA calculation
- Project & expense planning

	Pooches Pet Emporium				
Date	*Description*	*Dog*	*Income*	*Outgoings*	*Balance*
9/2/02	Fees – Mr C. Brown	Snoopy	96.37		96.37
10/2/02	Rubber bones			36.26	60.11
10/2/02	Fees – Mrs E. R. Windsor	7 corgis	1006.45		1066.56
12/2/02	Special order: 7 red carpets			47.28	992.28
16/2/02	Fees – Master T. Tin	Snowy	32.98		1025.26
17/2/02	Beefy Bruno's Bonemeal			243.47	781.79
21/2/02	Fees – Mr F. Flintstone	Dino	21.95		803.74
21/2/02	Special order: 1 Brontosaurus bone			6.47	797.27
28/2/02	Wages – Mr S. H. Ovelit			489.46	307.81

The WIMP interface

WIMP (Windows, Icons, Menus and Pointers)

- Currently many common environments for interactive computing are examples of the WIMP interface style, often simply called windowing systems.

- It is the default interface style for the majority of interactive computer sys- tems in use today, especially in the PC and desktop workstation arena.
- Examples ofWIMP interfaces include Microsoft Windows for IBM PC compatibles, MacOS for Apple Macintosh compatibles and various X Windows-based systems for UNIX.

Point-and-Click Interface

- User **points and clicks/taps** to perform actions
- Most actions require **single click or tap**
- Uses **icons, buttons, links, images**
- Closely related to **WIMP**, but **simpler**
- Strongly linked to **hypertext navigation**
- Works with **mouse and touchscreens**
- Often combined with **menu-driven interfaces**

Examples

Websites & web apps, Touchscreen kiosks, Google Maps (tap a place → details)

WIMP Interface Elements

Windows

- **Windows** act like **independent work areas c**an contain **text or graphics** support **move, resize, minimize, maximize a**llow **multitasking** (multiple tasks visible)
- It can be: **Overlapping** (one hides another), **Tiled** (side-by-side, no overlap), **Cascaded** (stacked with offset)
- It includes: **Title bar** (identification), **Scrollbars** (navigate large content), **Control buttons** (close, resize)

Examples

- **Browser tabs & windows**, **VS Code / IDE panels, Excel sheets inside app window, Split screen on laptops**, **Multitasking on tablets**

Why it is important for HCI:

- Shows how **interface design supports human multitasking**
- Reduces **cognitive load** by separating tasks
- Applies HCI principles: **Visibility, User control, Efficiency**
- Helps study **usability, attention switching & workflow design**

Icons

- **Icons** are small pictures representing **windows, files, apps, or actions**
- A window can be **iconified** (shrunk) to save screen space
- Iconified windows can be **restored by clicking**
- Help users **pause and resume tasks**
- Allow **many items** to be available at once

Examples

App icons on phones, **Recycle Bin / Trash**, **Minimized apps** on taskbar/dock, **Emoji-style icons** in apps, **Desktop shortcuts**

Why it is important for HCI:

- Supports **recognition over recall**
- Improves **usability & efficiency**
- Shows importance of **visual metaphors** in interface design

Menus

- Menus present a list of available actions
- Rely on recognition rather than recall
- Options must be clearly named & meaningful
- Items are highlighted when pointed at
- Selection via click or key press
- Large menus reduce efficiency → use cascading sub-menus
- Common menu types: Menu bar + pull-down menus Pop-up / context menus Cascading menus

Menus

Design Guidelines

- Avoid **too many items,** Group by **function,** Keep **frequent actions prominent**
- Separate **dangerous actions** (e.g., Delete vs Save)
- Maintain **consistency across applications**

Examples

- App menu bars, Right-click context menus, Three-dot (⋮) menus in mobile apps, Game option menus

Why it is important for HCI:

- Applies **recognition > recall**
- Uses **Fitts' Law** (target size & distance)
- Reduces **user errors**
- Supports **learnability & consistency**

Pointer

- **Pointer** enables **pointing, selecting, dragging,** controlled by **mouse, trackpad, joystick, trackball -** Shown as a **cursor** on the screen
- Cursor shape changes to show **mode or system status**
  - Arrow → normal
  - Crosshair → drawing
  - Spinner → system busy

Examples

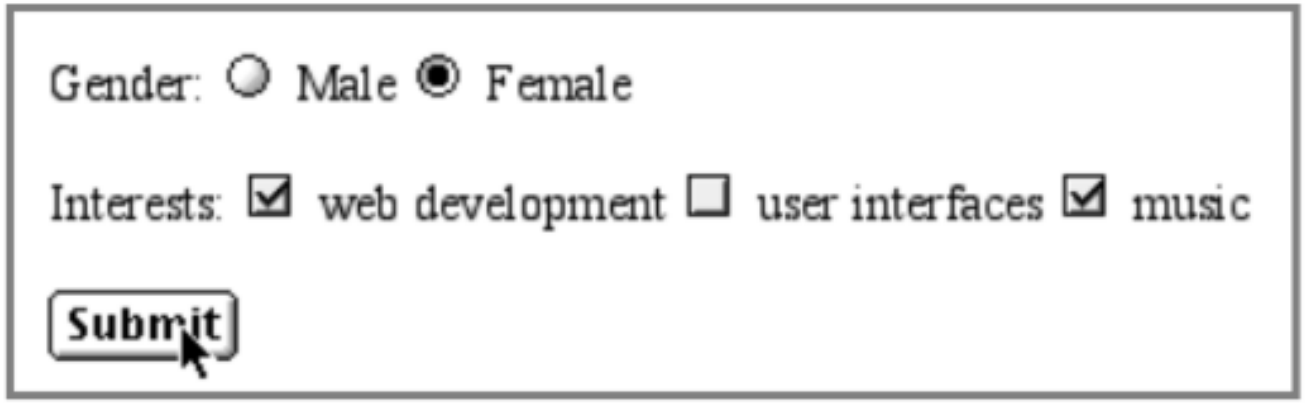
- Mouse pointer on laptops, **Touchpad cursor, Loading spinner** on apps, Drag-and-drop files

Why it is important for HCI:

- Core to **direct manipulation**
- Affects **speed, accuracy & user satisfaction**
- Demonstrates **feedback & affordance**
- Poor pointer design increases **user error**

Buttons

- Individual and isolated regions within a display that can be selected to invoke an action
- Special kinds (radio buttons, set of mutually exclusive choices, check boxes, set of non-exclusive choices)

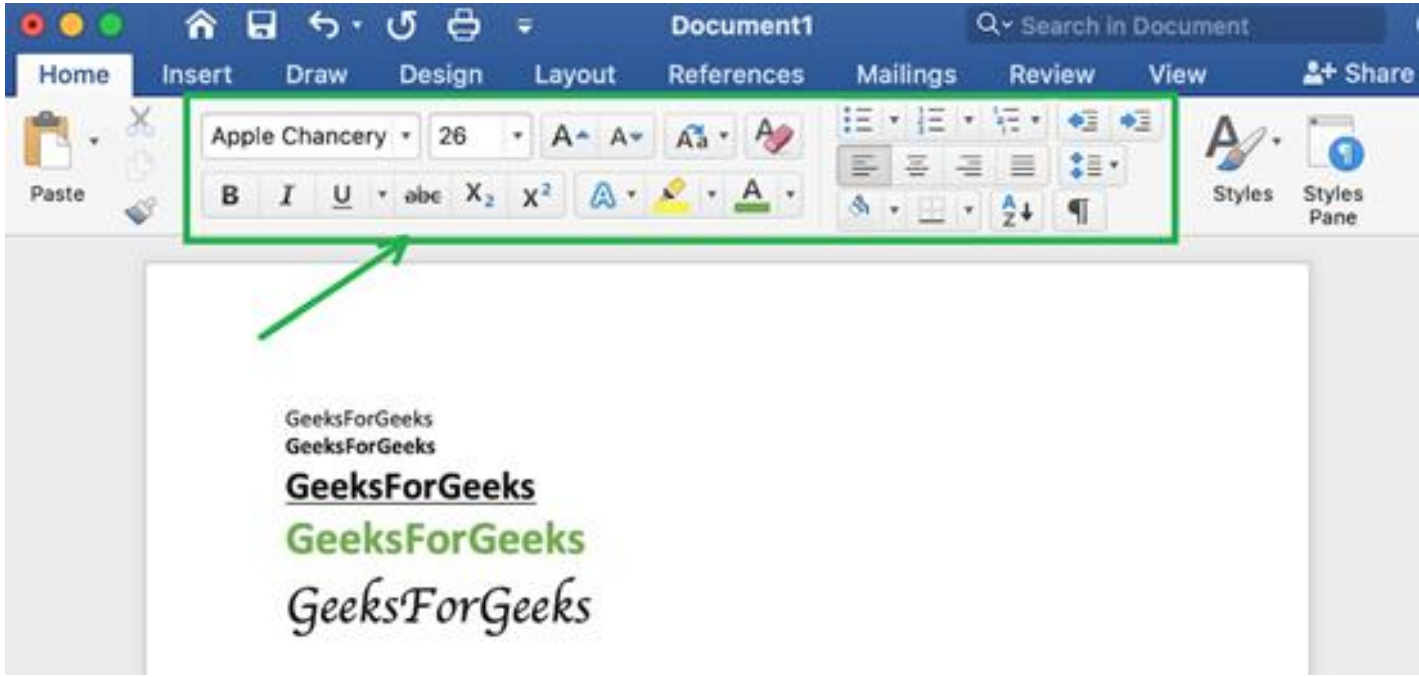


Toolbars

- long lines of icons ..... but what do they do?
- fast access to common actions
- often customizable:

  choose which toolbars to see

  choose what options are on it



Dialogue boxes

- Information windows that pop up to inform of an important event or request information.

Example: When saving a file, a dialogue box is displayed to allow the user to specify the filename and location. Once the file is saved, the box disappears.

Interactivity & Errors

- Errors are unavoidable
- Good interaction:
  - Helps users **detect errors**
  - Allows **easy correction**
- Success = ability to **recover**, not zero errors

Can you think of any examples?

Interactivity refers to how a system responds to user actions through feedback, confirmation, and error recovery, making interaction successful even when errors or inaccuracies occur.

Context of the Interaction

- Interaction happens within a context, not in isolation. Context includes:
  1. Physical environment
  2. Social factors (peers, managers)
  3. Organizational factors (work culture, policies)

- other people - desire to impress, competition, fear of failure
- Motivation - fear, allegiance, ambition, self-satisfaction
- inadequate systems - cause frustration and lack of motivation

Context of the Interaction

Examples

- Online exams with **live status feedback**
- Collaboration tools (Teams, Google Docs)
- Apps that show **loading indicators**

The context of interaction refers to the physical, social, and organizational factors that influence how users interact with a system and affect their motivation, performance, and satisfaction.

Experience, Engagement & Fun in HCI

- Traditional usability focused on:
- Effectiveness
- Efficiency
- Satisfaction
- Modern HCI goes beyond usability → experience matters
- Users are not machine parts; they choose what they want to use
- Especially important in: Entertainment, Shopping, Education, Leisure apps.

Understanding Experience

- Experience ≠ task completion (e.g., online buying vs “going shopping”)
- Experience includes; Emotion, Engagement, Social interaction
- State of deep engagement Occurs between: Boredom (too easy), Anxiety (too hard)
- Best interaction happens when users are: Challenged but capable

Virtual Christmas Crackers

Traditional cracker includes:

- Small toy
- Joke or motto
- Paper hat
- Pulling action
- Loud “bang”
- Shared experience
- The designers created a **virtual version**. But they didn’t copy it exactly. They redesigned the *experience*.

Deconstructing the Experience

Instead of copying the object, designers asked:

- What makes crackers fun?
- What emotions are involved?
- What social elements matter?
- What creates surprise?

This is called: **Experience Deconstruction**

Break down:

- Surface elements
- Emotional effects
- Social interaction
- Anticipation & suspense

Real vs Virtual Cracker (Comparison)

Experience Aspect	Real Cracker	Virtual Cracker
Design	Cheap & cheerful	Simple web graphics
Play	Plastic toy & joke	Web toy & joke
Dressing up	Paper hat	Printable mask
Shared	Offered physically	Sent by email/message
Hidden content	Inside cracker	Hidden until click
Suspense	Pulling tension	Slow page animation
Surprise	Bang!	Sound file (WAV)

Designing for Hiddenness & Suspense

Real cracker:

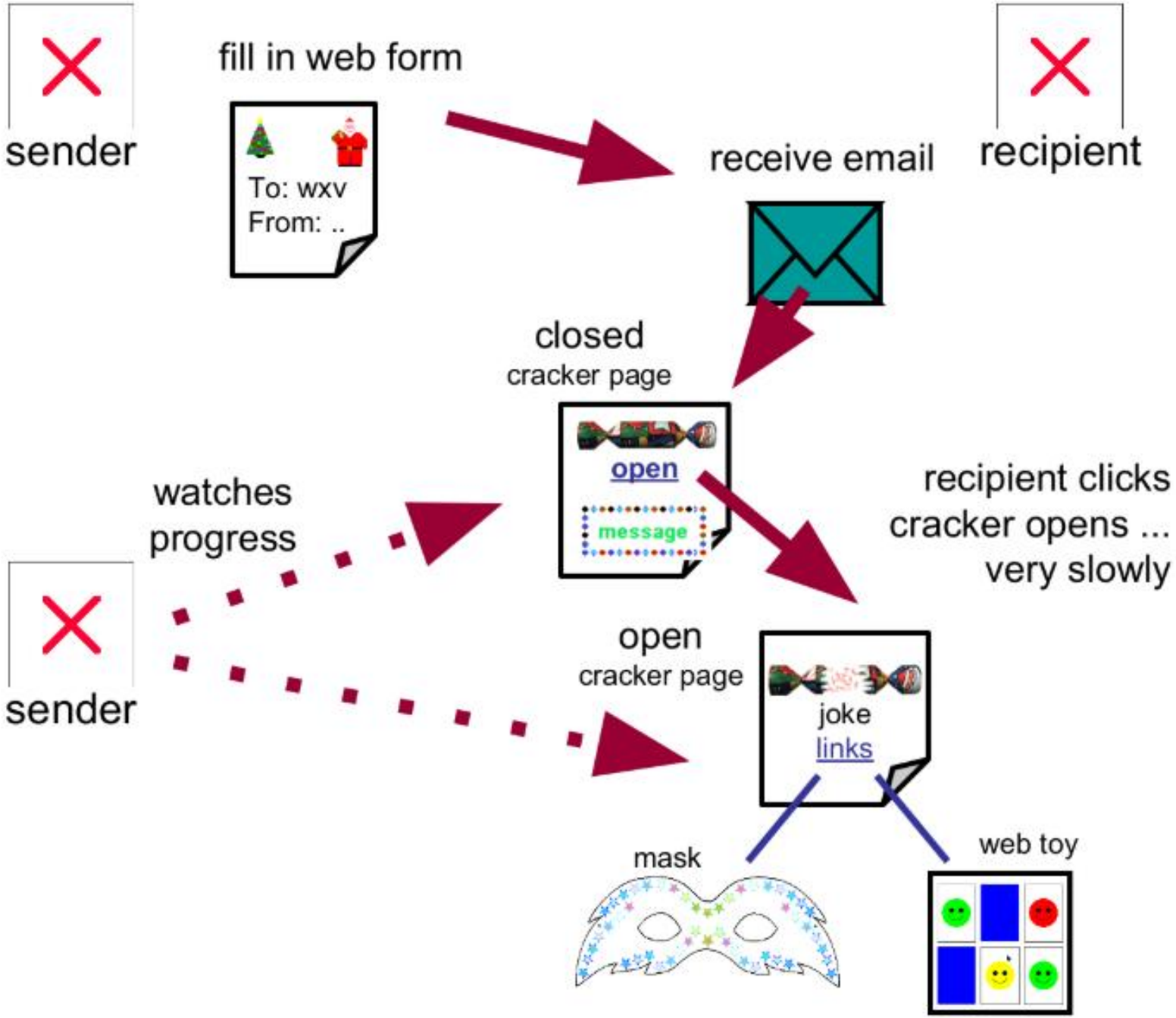
- You don’t know what’s inside
- Build tension while pulling
- Sudden “bang”

Virtual cracker:

- Sender cannot see content until opened
- First page shows only image
- User must click “Open”
- Slow animation builds suspense
- Sound effect replaces bang

HCI Lesson: Interaction design often replaces physical sensation with digital equivalents.

How virtual crackers work



How does the sender “watch progress”?

In the **virtual cracker design**, the sender can monitor the interaction through **system feedback mechanisms** built into the web application

When the recipient: Opens the email → Clicks the cracker link → Clicks “Open” → Fully opens the cracker

The system logs these actions on the server.

The sender’s interface then updates to show:

“Sent”→ “Opened” → “Cracker pulled” → “Viewed content”

This is similar to: WhatsApp ✓✓ read receipts, Instagram “Seen”

This creates a **co-experience**, even though users are not physically together.

Why Is This Important in HCI?

In a real cracker:

- Both people physically pull it.
- They experience the “bang” together.

In the virtual cracker:

- They are not co-located.
- The system recreates the shared moment digitally.

So the designer asked: How can we recreate shared anticipation remotely?

Answer: Through **real-time progress feedback to the sender**.

Physical Design & Engagement

Why Physical Design Still Matters

Even in a digital world:

- We still touch devices
- We press buttons
- We swipe screens
- We rotate dials
- We use controllers

Interaction begins with **physical manipulation**

Examples of Physical Controls

Microwave – smooth, easy-to-clean buttons

Washing machine – rotating dial

MiniDisc – multi-function knob

Each design fits its **context of use**.

Design Constraints

Designers don’t design freely. They face constraints:

- **Ergonomic:** Buttons must be big enough, Not too close together, Easy to press
- **Physical:** Device size limits control type, High-voltage switches cannot be tiny
- **Legal & Safety:** Cooker controls away from heat, Out of children’s reach
- **Context & Environment:** Microwave → smooth surface for cleaning
- **Aesthetic:** Must look attractive
- **Economic** Must be affordable

Smartphone Design

Consider smartphones:

- Thin body → less space for buttons
- Removed headphone jack → aesthetic + space trade-off
- Small side buttons → ergonomic compromise

Question:

- Would you prefer thicker phone with more controls?
- This is real-world constraint balancing.

Fluidity

How naturally physical movement matches logical function.

Good fluidity:

- Turn right → next track
- Turn left → previous track

Opposite physical actions → opposite logical outcomes That feels natural.

Gaming Controllers

Think about:

- Joystick forward → character moves forward
- Pull trigger → shoot
- Tilt phone → steering
- These work because of: Natural physical mapping
- If joystick forward made character go backward…
  Players would quit instantly.

Bad Mapping Example

Imagine:

- Volume up = rotate left
- Volume down = rotate right

This violates natural expectation.

Users feel:

- Frustration
- Cognitive load
- Loss of control
- Fluidity reduces mental effort.

Shannon Entropy

It measures **how uncertain or unpredictable something is**.

- In **Human–Computer Interaction (HCI)**, it is used to measure:
- How predictable user choices are
- How easy an interface is to use
- How quickly users can make decisions

Example idea:

- If a user has many equally likely choices → high uncertainty → high entropy
- If one choice is very obvious → low uncertainty → low entropy

So, entropy helps designers understand **how confusing or clear an interface is**.

Entropy Measures Uncertainty in User Decisions

- **Idea:** Entropy tells how **confused or unsure** a user is when choosing.
- **Example:** Imagine an ATM screen:
- **Case A (High Uncertainty):**

Options shown:

  - Service 1
  - Service 2
  - Service 3
  - Service 4

User doesn’t know what each means → must think → high entropy

Entropy Measures Uncertainty in User Decisions

Case B (Low Uncertainty):

- Options shown:
  - Withdraw Cash
  - Check Balance
  - Deposit Money
- User instantly understands → low entropy. So entropy measures how easy the decision is.

Helps Evaluate Menu Design Efficiency

Idea: Entropy helps designers check if a menu is **efficient or confusing**.

Example: A restaurant digital menu

Bad Design (High Entropy):

- 40 items in one long list
- No categories
- Similar names
- Customer takes long time to decide.

Good Design (Low Entropy):

Categories:

Burgers

Pizzas

Drinks

Best Sellers Highlighted, Customer Quickly chooses, Low Entropy

Used in Predictive Interfaces

Idea: Systems reduce entropy by **predicting what users want**.

Example: Smartphone keyboard, when you type:

"Good mor..."

The keyboard suggests:

- morning
- more
- move

Since "morning" is most likely, entropy is reduced because the system predicts your choice.

Another: YouTube recommendations show videos you are likely to watch → reducing uncertainty.

Lower Entropy = Faster Interaction

Idea: When entropy is low, users make decisions quickly.

Example: Website login:

High Entropy: Many login buttons:

- Sign in
- Access account
- Member portal
- Enter system

- User gets confused.

Low Entropy: Only one clear button: "Login"

User clicks instantly.

Less uncertainty → faster action.

Equal Probabilities = High Entropy

Idea: If all choices are equally likely, users must think more.

Example: Netflix homepage shows

- 20 random movies
- No categories
- No recommendations
- You don’t know what to pick → high entropy.

Mathematical Definition

$$H = -\sum p(x) \log_2 p(x)$$

The document gives the formula: $H = -\sum p(x) \log_2 p(x)$

What each part means:

H = Entropy (amount of uncertainty)

p(x) = Probability of each choice

$\log_2$ = Log base 2 (used because entropy is measured in **bits**)

Key idea:

- Entropy depends on **probabilities of choices**.
- Equal probabilities → Maximum entropy
- Unequal probabilities → Lower entropy

Numerical Example – Equal Probabilities

A software interface contains four menu options. First consider equal probabilities:

A = 0.25, B = 0.25, C = 0.25, D = 0.25

$$H = -\sum p(x) \log_2 p(x)$$

- H = - [4 × (0.25 logn 0.25)]
- logn 0.25 = -2
- H = - [4 × (0.25 × -2)] = 2 bits
- This represents high uncertainty and slower decision-making.

Numerical Example - UnEqual Probabilities

Now consider unequal probabilities:

A = 0.70, B = 0.10, C = 0.10, D = 0.10

H = - [0.70 logn 0.70 + 3(0.10 logn 0.10)]

logn 0.70 » -0.515, logn 0.10 » -3.322

H » 1.36 bits

This represents lower uncertainty and faster interaction.

STEP by STEP calculation

We are given probabilities:

- A = 0.70
- B = 0.10
- C = 0.10
- D = 0.10

Formula:

$$H = -\sum p(x) \log_2 p(x)$$

Since three values are the same (0.10), we can write:

$$H = -[0.70 \log_2 0.70 + 3(0.10 \log_2 0.10)]$$

We use **log base 2** because entropy is measured in bits.

Given:

$$\log_2(0.70) \approx -0.515$$

$$\log_2(0.10) \approx -3.322$$

First term:

$$0.70 \times (-0.515) = -0.3605$$

Second term:

$$0.10 \times (-3.322) = -0.3322$$

Since there are **3 such terms**:

$$3 \times (-0.3322) = -0.9966$$

−0.3605+(−0.9966)=−1.3571

H=−(−1.3571)

$$H = 1.3571$$

H≈1.36 bits

Activity 4

https://tinyurl.com/Spring2026-Activity4

Solution Question 1

Design A: H = 2 bits

Design B: H=−[0.85(−0.234)+3(0.05(−4.322))]

$$= 0.199 + 0.648$$

$$H \approx 0.85 \textit{ bits}$$

Design B is much better because:

- Very low entropy
- Users quickly find main option

Solution Question 2

Design A

Log values: $\log_2(0.60) = -0.737$ $\log_2(0.20) = -2.322$
$\log_2(0.10) = -3.322$

Multiply terms: $0.60 \times (-0.737) = -0.442$ $0.20 \times (-2.322) = -0.464$ $0.10 \times (-3.322) = -0.332$

Two such terms: $2 \times -0.332 = -0.664$

Add $-0.442 - 0.464 - 0.664 = -1.57$

Final entropy $H_A = 1.57$ *bits*

Solution Question 2

Design A

Log values: $\log_2(0.60) = -0.737$ $\log_2(0.20) = -2.322$ $\log_2(0.10) = -3.322$

Multiply terms: $0.60 \times (-0.737) = -0.442$ $0.20 \times (-2.322) = -0.464$ $0.10 \times (-3.322) = -0.332$

Two such terms: $2 \times -0.332 = -0.664$

Add $-0.442 - 0.464 - 0.664 = -1.57$

Final entropy $H_A = 1.57$ *bits*

Solution Question 2

Design B

Log values: $\log_2(0.40) = -1.322$ $\log_2(0.30) = -1.737$ $\log_2(0.20) = -2.322$ $\log_2(0.10) = -3.322$

Multiply terms:

- $0.40 \times -1.322 = -0.529$
- $0.30 \times -1.737 = -0.521$
- $0.20 \times -2.322 = -0.464$
- $0.10 \times -3.322 = -0.332$

$-0.529 - 0.521 - 0.464 - 0.332 = -1.846$

Design	Entropy
Design A	**1.57 bits**
Design B	**1.85 bits**

Final entropy $H_B = 1.85\ bits$